

6. Tablas de símbolos para agregaciones de datos

- Definición y conceptos
- Patrones

Definición y conceptos

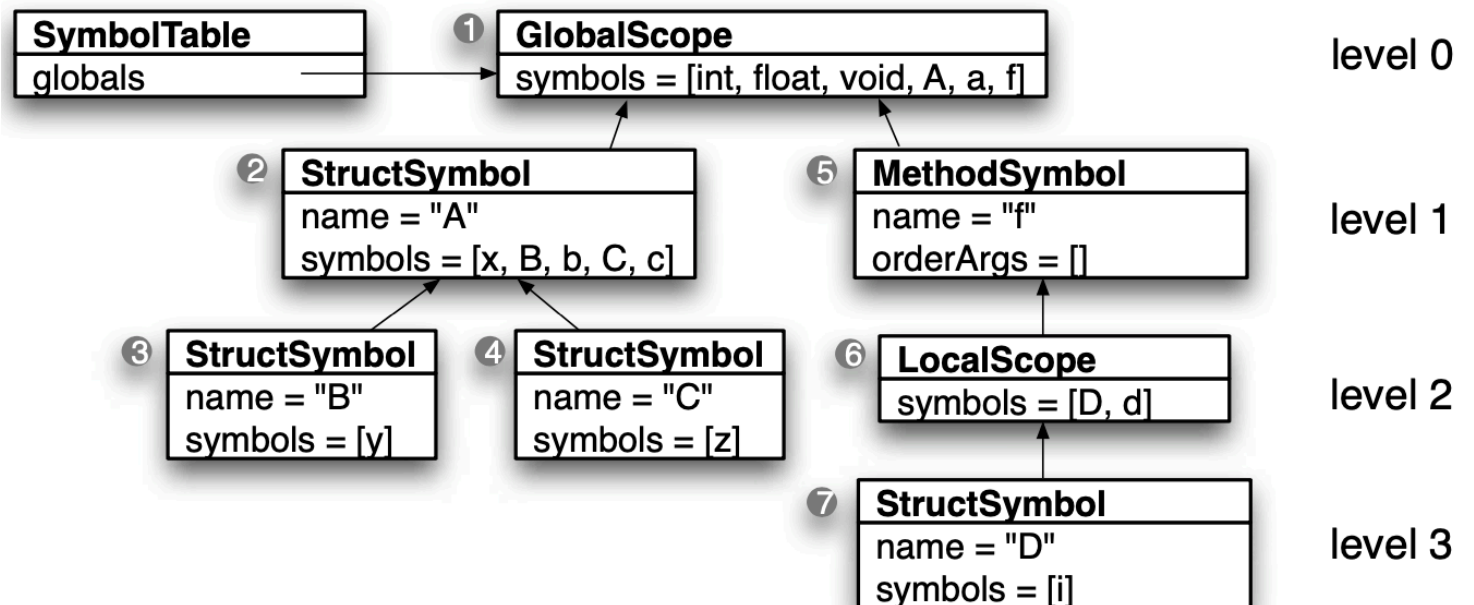
- En la unidad anterior se revisó lo básico sobre tablas de símbolos: definición de símbolos, agrupaciones y su organización en alcances.
- En esta unidad se revisará otro tipo de alcance llamado alcance para agregaciones de datos.
- Como cualquier otro alcance, contiene símbolos y tiene su lugar dentro del árbol de alcances. La diferencia es que el código fuera de la agregación de datos puede acceder a los miembros utilizando una notación **user.name**.

Alcance para estructuras

```
① // start of global scope
② struct A {
    int x;
③ struct B { int y; };
    B b;
④ struct C { int z; };
    C c;
};
A a;
```

- Los alcances para estructuras se comportan como alcances locales desde el punto de vista de su construcción.

```
⑤ void f()
⑥ {
⑦ struct D {
    int i;
};
D d;
d.i = a.b.y;
}
```



Alcance para estructuras

- Dentro del alcance de la estructura, resolvemos los símbolos revisando en el árbol de alcances como cualquier otro alcance anidado.
- También se debe resolver símbolos dentro de la estructura desde fuera de ella, un traductor debe poder reconocer a que se refiere un enunciado como **exp.x**.

Patrones - Tabla de símbolos para estructuras

Propósito.

- Este patrón rastrea símbolos y construye un árbol de alcance para lenguajes con agregaciones de datos tipo estructuras (structs).

Discusión.

- Para manejar alcance de estructuras, se construye un árbol de alcance y se definen símbolos como se hizo en el patrón Tabla de símbolos para alcance anidado. La única diferencia se encuentra en la resolución de símbolos ya que expresiones como a.b pueden ver los campos dentro de una estructura.

Patrones - Tabla de símbolos para estructuras

- A continuación se muestran las reglas para construir un árbol de alcance a partir de alcances anidados y resolver símbolos en el contexto semántico correcto.

Upon

Start of file

Variable declaration x

struct declaration S

Method declaration f

{
}

Variable reference x

Member access «*expr*». x

End of file

Action(s)

push a GlobalScope. def BuiltInType objects for int, float, void.

ref x 's type. def x as a VariableSymbol object in the current scope. This works for globals, **struct** fields, parameters, and locals.

def S as a StructSymbol object in the current scope and push it as the current scope.

ref f 's return type. def f as a MethodSymbol object in the current scope and push it as the current scope.

push a LocalScope as the new current scope.
pop, revealing the previous scope as the current scope. This works for **structs**, methods, and local scopes.

ref x starting in the current scope. If not found, look in the immediately enclosing scope if any.

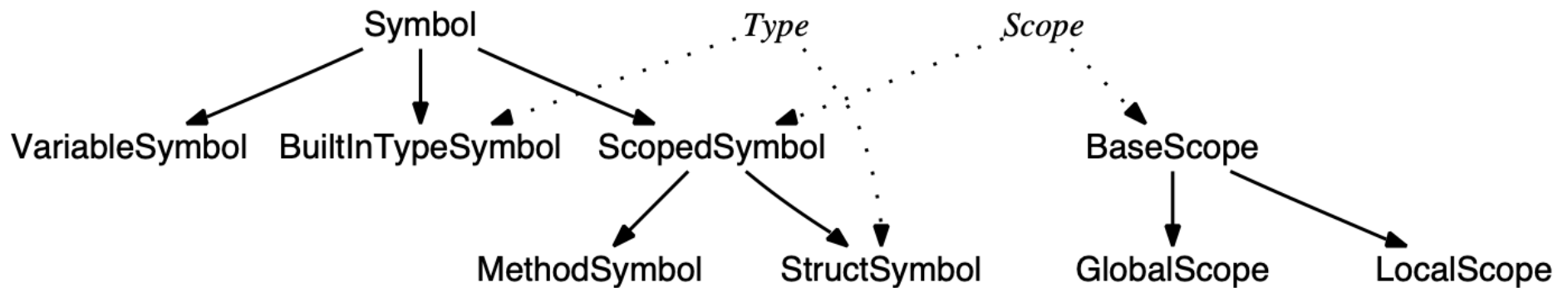
Compute the type of «*expr*» using the previous rule and this one recursively. Ref x only in that type's scope, not in any enclosing scopes.

pop the GlobalScope.

Patrones - Tabla de símbolos para estructuras

Implementación.

- Este patrón puede utilizar como base el patrón Tabla de símbolos para alcance anidado y agregar a la jerarquía las clases que complementen.
- Se necesita un nuevo tipo de nodo para el árbol de alcance, llamado StructSymbol, que represente las estructuras dentro del programa.



Patrones - Tabla de símbolos para estructuras

```
public class Symbol {  
    String name;  
    Type type;  
    Scope scope;  
  
    public Symbol(String name) {  
        this.name = name;  
    }  
  
    public Symbol(String name, Type type) {  
        this(name);  
        this.type = type;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public String toString() {  
        if (type!=null)  
            return '<' + getName() + ":" + type + '>';  
  
        return getName();  
    }  
}
```

- El código modificado para la clase Symbol.

Patrones - Tabla de símbolos para estructuras

```
public abstract class ScopedSymbol extends Symbol implements Scope {
    3 usages
    Scope enclosingScope;

    1 usage
    public ScopedSymbol(String name, Type type, Scope enclosingScope) {
        super(name, type);
        this.enclosingScope = enclosingScope;
    }

    1 usage
    public ScopedSymbol(String name, Scope enclosingScope) {
        super(name);
        this.enclosingScope = enclosingScope;
    }

    7 usages
    public Symbol resolve(String name) {
        Symbol s = getMembers().get(name);

        if ( s!=null )
            return s;

        // if not here, check any enclosing scope
        if ( getEnclosingScope() != null ) {
            return getEnclosingScope().resolve(name);
        }

        return null; // not found
    }
}
```

- El código para la clase ScopedSymbol.

Patrones - Tabla de símbolos para estructuras

```
public Symbol resolveType(String name) {  
    return resolve(name);  
}
```

7 usages

```
public void define(Symbol sym) {  
    getMembers().put(sym.name, sym);  
    sym.scope = this; // track the scope in each symbol  
}
```

2 usages

```
public Scope getEnclosingScope() {  
    return enclosingScope;  
}
```

```
public String getScopeName() {  
    return name;  
}
```

```
/** Indicate how subclasses store scope members. Allows us to  
 * factor out common code in this class.  
 */
```

2 usages 2 implementations

```
public abstract Map<String, Symbol> getMembers();
```

```
}
```

- El código para la clase ScopedSymbol.

Patrones - Tabla de símbolos para estructuras

```
public class StructSymbol extends ScopedSymbol implements Type, Scope {  
    Map<String, Symbol> fields = new LinkedHashMap<String, Symbol>();
```

```
    public StructSymbol(String name, Scope parent) {  
        super(name, parent);  
    }
```

- El código para la clase StructSymbol.

```
    public Symbol resolveMember(String name) {  
        return fields.get(name);  
    }
```

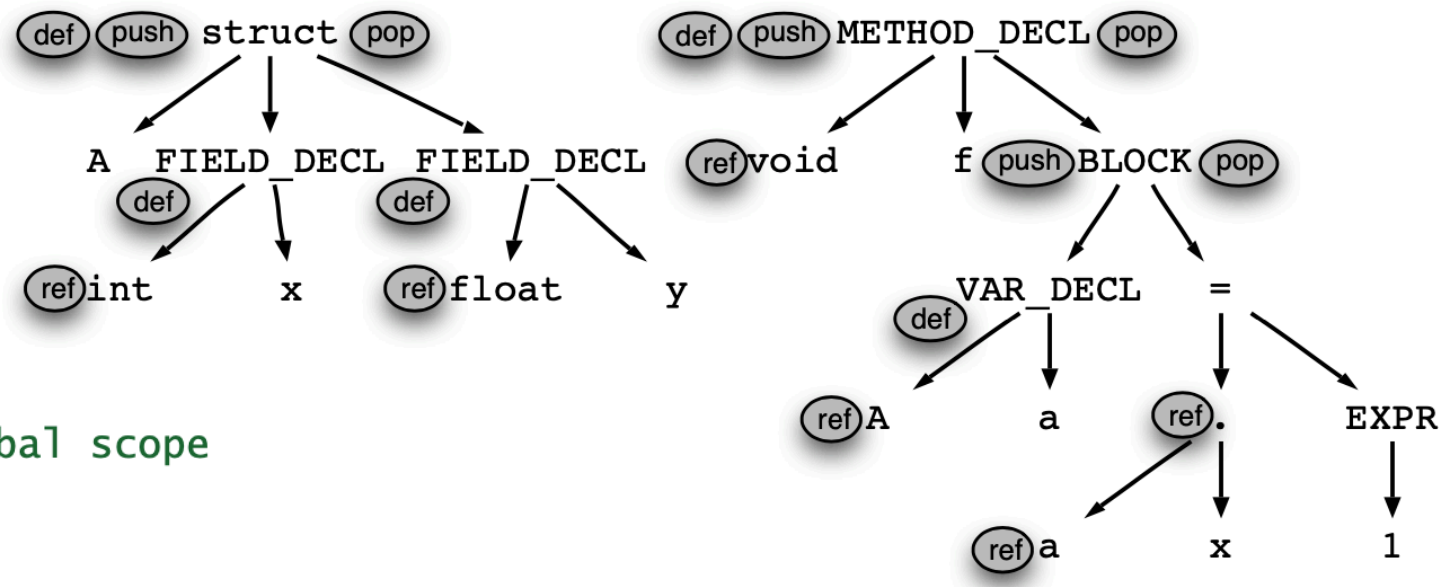
```
    public Map<String, Symbol> getMembers() {  
        return fields;  
    }
```

```
    public String toString() {  
        return "struct " + name + ":{ " + fields.keySet().toString() + " }";  
    }
```

```
}
```

Patrones - Tabla de símbolos para estructuras

- En la figura se muestra el árbol de sintaxis para procesar un struct.



```
① // start of global scope
② struct A {
    int x;
    float y;
};
③ void f()
④ {
    A a; // define a new A struct called a
    a.x = 1;
}
```

```

public class AlcanceStruct {
    public static void main(String[] args) {
        Scope currentScope;

        currentScope = new GlobalScope();

        currentScope.define(new BuiltInTypeSymbol("int"));
        currentScope.define(new BuiltInTypeSymbol("float"));
        currentScope.define(new BuiltInTypeSymbol("void"));

        // struct A
        StructSymbol ss = new StructSymbol("A", currentScope);
        currentScope.define(ss);
        currentScope = ss;

        // int x
        BuiltInTypeSymbol t = (BuiltInTypeSymbol) currentScope.resolve("int");
        if (t == null) {} // Excepcion
        currentScope.define(new VariableSymbol("x", t));

        // float y
        t = (BuiltInTypeSymbol) currentScope.resolve("float");
        if (t == null) {} // Excepcion
        currentScope.define(new VariableSymbol("y", t));

        currentScope = currentScope.getEnclosingScope(); // Sale del struct
    }
}

```

```

① // start of global scope
② struct A {
    int x;
    float y;
};
③ void f()
④ {
    A a; // define a new
    a.x = 1;
}

```

```
// void g()
BuiltInTypeSymbol rt = (BuiltInTypeSymbol) currentScope.resolve("void");
if (rt == null) {} // Excepcion
```

```
MethodSymbol m = new MethodSymbol("f", null, currentScope);
currentScope.define(m);
currentScope = m;
```

```
currentScope = new LocalScope(currentScope);
```

```
// A a
ss = (StructSymbol) currentScope.resolve("A");
if (ss == null) {} // Excepcion
```

```
currentScope.define(new VariableSymbol("a", ss));
```

```
//a.x = 1
```

```
VariableSymbol v = (VariableSymbol) currentScope.resolve("a");
ss = (StructSymbol) v.type;
v = (VariableSymbol) ss.resolveMember("x");
```

```
if (v == null) {} // Excepcion
```

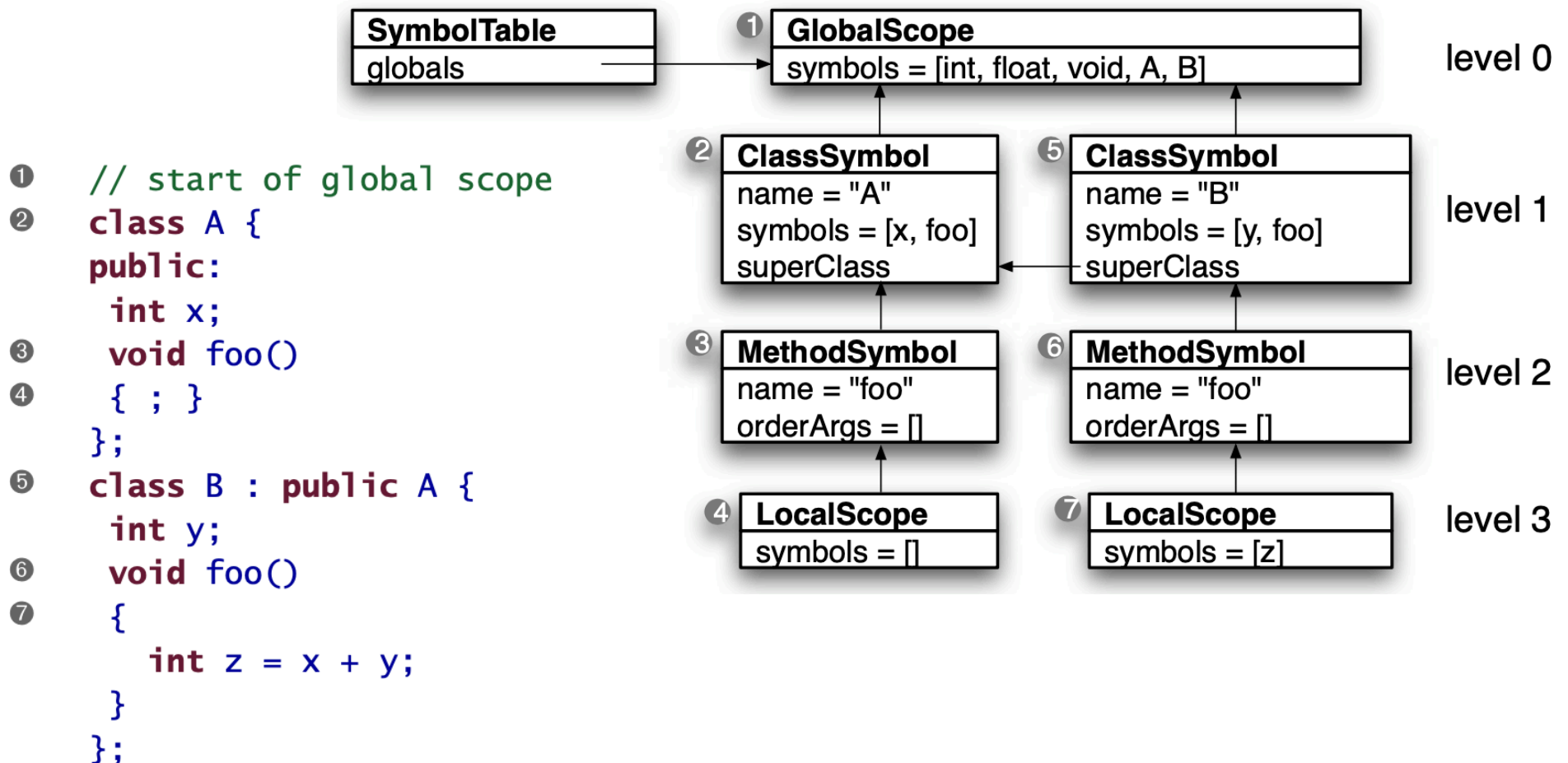
```
currentScope = currentScope.getEnclosingScope(); // Sale del bloque
currentScope = currentScope.getEnclosingScope(); // Sale de f
```

```
① // start of global scope
② struct A {
    int x;
    float y;
};
③ void f()
④ {
    A a; // define a new A s
    a.x = 1;
}
```

```
}
```

Alcance para clases

- Los alcances para clases son como los de estructuras, pero con herencia. Esto significa que las clases tienen dos alcances padre: el alcance dentro del cual se encuentran y un alcance superclase.



Alcance para clases

- Los apuntadores hacia los alcances anidados apuntan hacia arriba como en las tablas de símbolos para estructuras, pero los apuntadores de superclase apuntan horizontalmente, porque todas las clases no anidadas, viven en el mismo nivel de alcance.

Alcance para clases

- Del mismo modo que con los **structs**, también se deben resolver las expresiones de acceso a los miembros, de manera diferente.

```
int g;                // global variable g
class A {
public:
    int x;
    void foo() { g = 1; } // can see global g
};

int main() {
    A a;
    a.x = 3; // no problem; x is the field of A
    a.g = 3; // ERROR! Should not see global variable g
}
```

Alcance para clases

- Las clases también permiten referencias hacia adelante a métodos, tipos o variables definidas posteriormente en el archivo de entrada.

```
class A {  
    void foo() { x = 3; } // forward reference to field x  
    int x;  
};
```

- Para manejar estas referencias hacia adelante, se podría intentar buscar definiciones futuras, pero hay una forma más fácil. Se pueden hacer dos pasadas sobre la entrada, una para definir símbolos y otra para resolverlos. Es decir, en una pasada se definiría **A**, **foo** y **x**. Después en la segunda pasada se analizará de nuevo la entrada buscando referencias.

Patrones - Tabla de símbolos para clases

Propósito.

- Este patrón rastrea símbolos y construye un árbol de alcance para lenguajes con clases no anidadas con herencia sencilla.

Discusión.

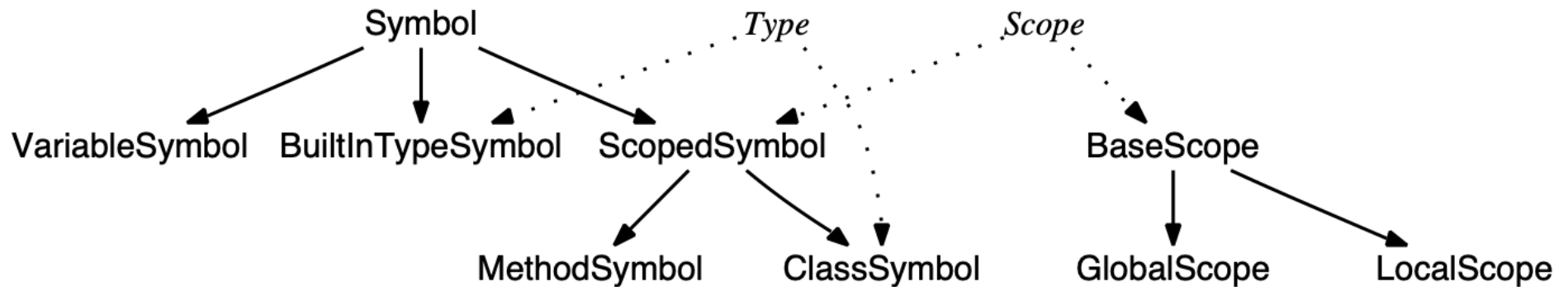
- Las clases son agregaciones de datos que permiten métodos y pueden heredar de superclases. Los métodos dentro de las clases pueden ver las variables globales. Para apoyar la semántica de este tipo de lenguajes, necesitamos modificar el árbol de alcances que se utilizó para las estructuras en el patrón anterior.

Patrones - Tabla de símbolos para clases

- Las clases son agregaciones de datos que permiten miembros métodos y pueden heredar miembros de superclases.
- En algunos lenguajes de programación orientada a objetos, los métodos pueden ver variables globales.
- Para apoyar la semántica de este tipo de lenguajes, es necesario modificar el patrón Tabla de símbolos para estructuras. Se reemplazará la clase StructSymbol con ClassSymbol y se agregará un apuntador a su superclase, además del que ya tenía para el alcance (enclosing scope).

Patrones - Tabla de símbolos para clases

- Se necesita un nuevo tipo de nodo para el árbol de alcance, llamado **ClassSymbol**, que represente las clases dentro del programa.



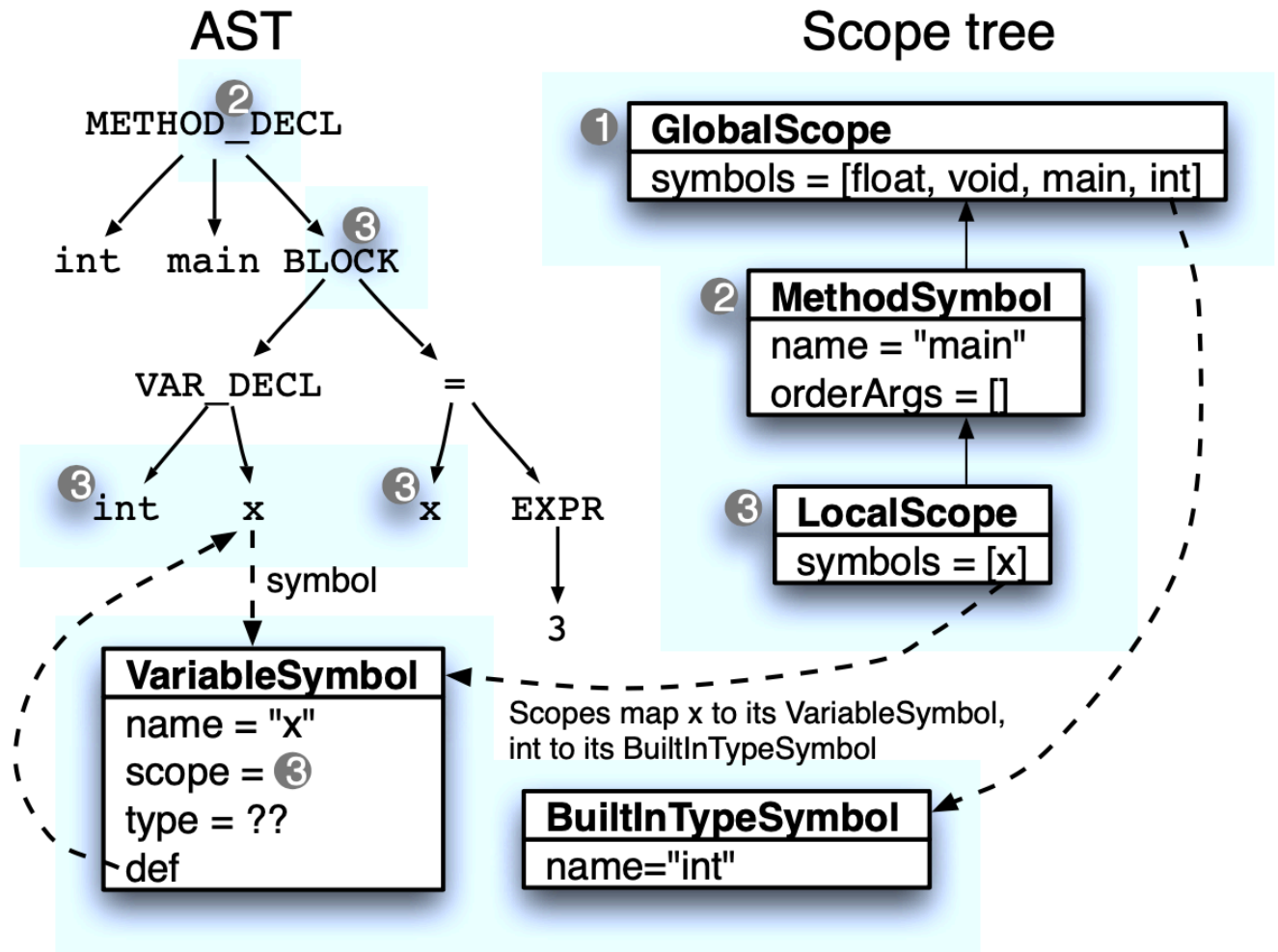
Patrones - Tabla de símbolos para clases

- Para implementar tablas de símbolos para lenguajes orientados a objetos que permiten referencias hacia adelante, es necesario extender el algoritmo para dar 2 pasadas, una para definir y otra para resolver los símbolos.
- Al separar en 2 fases se introduce un problema de comunicación.
- Para resolver las referencias a los símbolos, necesitamos el alcance actual, pero éste se calcula en la fase de definición. Es necesario pasar información entre las 2 fases. El lugar lógico para hacer esto es el árbol de sintaxis.
- La fase de definición da seguimiento al alcance actual (current scope) y lo registra en los nodos correspondientes.
- Por ejemplo, cuando se encuentra una { se registra el alcance local asociado (LocalScope) y conforme se van definiendo símbolos, se deben registrar también en el árbol.
- La fase de resolución puede ver en el árbol la información del alcance y los símbolos ya definidos.

Patrones - Tabla de símbolos para clases

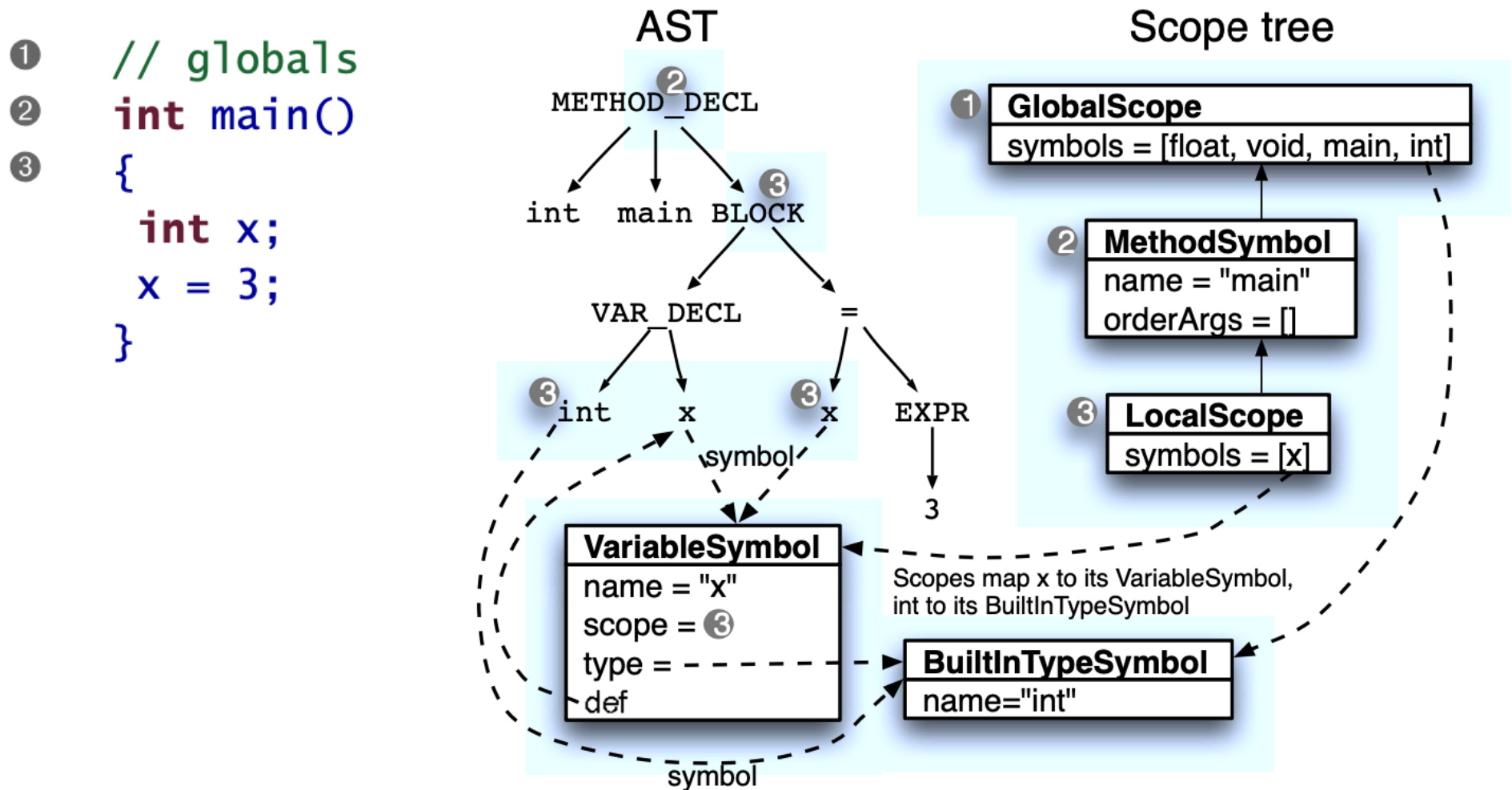
- Fase de definición.

```
① // globals
② int main()
③ {
    int x;
    x = 3;
}
```



Patrones - Tabla de símbolos para clases

- Fase de resolución.



Patrones - Tabla de símbolos para clases

```
public Symbol resolve(String name) {  
    Symbol s = members.get(name);    // look in this scope  
    if ( s!=null ) return s;         // return it if in this scope  
    if ( getParentScope() != null ) { // do we have a super or enclosing?  
        return getParentScope().resolve(name); // check super/enclosing  
    }  
    return null; // not found in this scope or above  
}
```

- La diferencia en el método resolve es que se invoca getParentScope(), no getEnclosingScope().
- Para decidir cual método invocar, depende del alcance que se esté analizando.

Patrones - Tabla de símbolos para clases

```
/** Where to look next for symbols; superclass or enclosing scope */  
public Scope getParentScope();  
/** Scope in which this scope defined. For global scope, it's null */  
public Scope getEnclosingScope();
```

- La interfaz Scope debe declarar estos 2 métodos.
- Para todas las clases que implementen Scope, excepto ClassSymbol, getParentScope() debe regresar enclosingScope. El método getParentScope() en las clases debe regresar el apuntador a la superclase. Si no hay superclase, debe regresar el apuntador enclosingScope.

Patrones - Tabla de símbolos para clases

- A continuación se muestran las reglas para construir un árbol de alcance para la fase de definición.

Upon	Action(s)
Start of file	push a GlobalScope. def BuiltInType objects for int, float, void.
Identifier reference x	Set x's scope field to the current scope (the resolution phase needs it).
Variable declaration x	def x as a VariableSymbol object, sym, in the current scope. This works for globals, class fields, parameters, and locals (wow). Set sym.def to x's ID AST node. Set that ID node's symbol to sym. Set the scope field of x's type AST node to the current scope.
Class declaration C	def C as a ClassSymbol object, sym, in the current scope and push it as the current scope. Set sym.def to the class name's ID AST node. Set that ID node's symbol to sym. Set the scope field of C's superclass' AST node to the current scope.
Method declaration f	def f as a MethodSymbol object, sym, in the current scope and push it as the current scope. Set sym.def to the function name's ID AST node. Set that ID node's symbol to sym. Set the scope field of f's return type AST node to the current scope.
{	push a LocalScope as the new current scope.
}	pop, revealing previous scope as current scope.
End of file	pop the GlobalScope.

Patrones - Tabla de símbolos para clases

- A continuación se muestran las reglas para construir un árbol de alcance para la fase de resolución.

Upon	Action(s)
Variable declaration x	Let t be the ID node for x 's type. ref t , yielding sym. Set t .symbol to sym. Set x .symbol.type to sym; in other words, jump to the VariableSymbol for x via the AST node's symbol field and then set its type field to sym.
Class declaration C	Let t be the ID node for C 's superclass. ref t , yielding sym. Set t .symbol to sym. Set C 's superclass field to sym.
Method declaration f	Let t be the ID node for f 's return type. ref t , yielding sym. Set t .symbol to sym. Set the type field of the MethodSymbol for f to sym.
Variable reference x	ref x , yielding sym. Set x .symbol to sym.
this	Resolve to the surrounding class scope. Set the symbol field of this's ID node to the surrounding class scope.
Member access « $expr$ ». x	Resolve « $expr$ » to a particular type symbol, esym, using these rules. ref x within esym's scope, yielding sym. Set x .symbol (x 's ID node) to sym.

Patrones - Tabla de símbolos para clases

- Para resolver el acceso a miembros, por ejemplo «**expr**».x, se hace de manera similar. La única diferencia es que la resolución debe parar al nivel de la raíz de la jerarquía de clases, no se debe buscar en el alcance global.

```
public Symbol resolveMember(String name) {  
    Symbol s = members.get(name);  
    if ( s!=null ) return s;  
    // if not here, check superclass chain only  
    if ( superClass != null ) {  
        return superClass.resolveMember(name);  
    }  
    return null; // not found  
}
```

Patrones - Tabla de símbolos para clases

Symbol	Reference	Resolution Algorithm	Scope Tree Walk
x	in method	Look first in the enclosing local scope(s), then into the method scope, and then into the enclosing class scope. If not found in the enclosing class, look up the class hierarchy. If not found in the class hierarchy, look in the global scope.	
x	in field definition initialization expression	Look in the surrounding class scope. If not found in that class, look up the class hierarchy. If not found in the class hierarchy, look in the global scope.	
x	in global scope	Look in the surrounding global scope.	

Patrones - Tabla de símbolos para clases

- Las reglas y algoritmos de resolución nos dicen todo lo necesario para construir una tabla de símbolos para un lenguaje orientado a objetos con herencia sencilla.
- También nos dice como resolver símbolos apropiadamente, tomando en cuenta las cadenas de jerarquía de clases y alcances.

Patrones - Tabla de símbolos para clases

Implementación.

- Para implementar este patrón se utiliza como base el patrón para estructuras. La principal diferencia es que se hacen dos fases sobre el árbol de sintaxis.
- Se deb implementar ClassSymbol para almacenar la información de las clases en el programa.

```
public class ClassSymbol extends ScopedSymbol implements Scope, Type {  
    /** This is the superclass not enclosingScope field. We still record  
     * the enclosing scope so we can push in and pop out of class defs.  
     */  
    ClassSymbol superClass;  
    /** List of all fields and methods */  
    public Map<String,Symbol> members=new LinkedHashMap<String,Symbol>();  
}
```


Patrones - Tabla de símbolos para clases

- Se tiene que definir el método getParentScope(), el cual regresa por omisión lo mismo que getEnclosing Scope(), pero para las clases regresa la superclase.

```
public Scope getParentScope() { return getEnclosingScope(); }
```

```
public Scope getParentScope() {  
    if ( superClass==null ) return enclosingScope; // globals  
    return superClass; // if not root object, return super  
}
```

Patrones - Tabla de símbolos para clases

- Si se declaran clases anidadas, se requiere conocer la referencia a la clase principal.

```
/** 'this' and 'super' need to know about enclosing class */  
public static ClassSymbol getEnclosingClass(Scope s) {  
    while ( s!=null ) { // walk upwards from s looking for a class  
        if ( s instanceof ClassSymbol ) return (ClassSymbol)s;  
        s = s.getParentScope();  
    }  
    return null;  
}
```